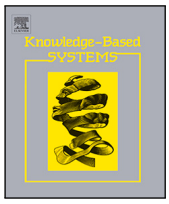
Knowledge-Based Systems 311 (2025) 113049

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys



Long-term evolutionary patterns matter: Self-supervised anomaly detection on dynamic graphs

Yun Fu [a], Che Zhou [a], Jintang Li [b], Liang Chen [a,*]

[a] School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China
[b] School of Software Engineering, Sun Yat-sen University, Zhuhai, China

ARTICLE INFO

Keywords:
Anomaly detection
Dynamic graphs
Contrastive learning

ABSTRACT

Graph anomaly detection (GAD) plays a crucial role in identifying anomalous individuals (e.g., nodes or edges) whose behaviors or patterns deviate significantly from the normal majority within a graph. Recent years have witnessed breakthrough advancements in research on static graphs. However, the GAD problem becomes more challenging when switching from static graphs to dynamic graphs due to the temporal evolution of graph structures and the scarcity of anomaly labels. Although various approaches have been proposed to address these challenges, they often struggle to capture the evolving dynamics and neglect the potential of capturing nodes' smooth long-term evolutionary patterns for self-supervised anomaly detection in dynamic graph. In this work, we empirically demonstrate that normal nodes exhibit smooth long-term evolutionary patterns, while anomalous nodes deviate significantly from their long-term histories. Motivated by this observation, we propose a multi-step histories-based contrastive learning framework, MHisCL, to detect anomalies in dynamic graphs in a self-supervised manner. Specifically, we treat a node's current state and its multi-step historical states as a positive contrastive pair, encouraging alignment between them to capture the smooth long-term evolutionary patterns of normal nodes. Consequently, MHisCL is capable of efficiently detecting anomalies by measuring the disagreement of nodes' positive pairs through a simple similarity computation. Furthermore, to better capture complex evolving dynamics, MHisCL incorporates a novel state space time encoding (SSTE), which explicitly models the temporal evolution of node representations through a linear dynamical system. We evaluate MHisCL on several dynamic graph benchmarks, and it outperforms state-of-the-art baseline by large margins.

1. Introduction

Anomaly detection has long been a classical problem in academia and industry. The goal of anomaly detection is to uncover unusual occurrences, outliers, or suspicious activities that may indicate fraud, errors, or anomalies in data. As instances that appear in real-world scenarios are often have rich relationships with each other, and can be naturally represented with graphs, graph anomaly detection (GAD) have garnered substantial attention and achieved breakthrough advancements in fraud detection [1], transaction network [2], and communication network [3] etc. Most existing graph anomaly detection methods focus on static graphs [4–7], where nodes and edges are stationary and do not evolve over time. However, many real-world graphs are intrinsically dynamic, characterized by frequent updates to nodes, edges, and attributes, termed *temporal graphs* or *dynamic graphs* in literature [8]. This evolving nature of graph structures makes anomaly detection in dynamic graphs more challenging.

Recently, various methods have been proposed to address this challenging problem. For example, Netwalk [9] utilizes a random walks-based autoencoder to learn node embeddings and perform dynamic clustering to identify anomalous edges. AddGraph [10] utilizes graph convolutional networks (GCNs) and Gated Recurrent Units (GRUs) to encode structural and temporal information, respectively, and trains an edge anomaly detector with link prediction as the proxy task to detect anomalous edges in an end-to-end manner. TADDY [2] use a transformer encoder with informative spatial–temporal node encodings to simultaneously capture structural and temporal information. However, due to the scarcity of anomaly labels in real-world scenarios, these link prediction-based methods [2,10,11] often resort to constructing pseudo anomalous edges for training, e.g., randomly sampling from non-existent edges. As a result, these methods suffer from noisy labels and are prone to performance degradation. To address

* Corresponding author.
E-mail addresses: fuyun5@mail2.sysu.edu.cn (Y. Fu), zhouch76@mail2.sysu.edu.cn (C. Zhou), lijt55@mail2.sysu.edu.cn (J. Li), chenliang6@mail.sysu.edu.cn (L. Chen).

https://doi.org/10.1016/j.knosys.2025.113049
Received 20 October 2024; Received in revised form 11 January 2025; Accepted 16 January 2025
Available online 23 January 2025

this issue, RustGraph [12] introduces a variational structural-temporal graph auto-encoder to learn node embeddings and generate more robust pseudo-labels based on the reconstructed adjacency matrix. FALCON [13] introduces an attention alignment contrastive learning to enhance the model's generalization capabilities.

Despite these advancements, these methods are primarily designed for discrete-time dynamic graphs (DTDGs), where a dynamic graph is represented as a sequence of static graph snapshots. Consequently, these approaches are limited to capturing coarse-grained temporal dependencies among snapshots, failing to model the more fine-grained temporal dynamics at the edge level. In response, recent methods have shifted attention toward continuous-time dynamic graphs (CTDGs), where a dynamic graph is represented as edge event streams with precise timestamps. For example, SAD [1] employs functional time encoding (FTE) [14] to encode the fine-grained time span between edges and utilizes historical predicted anomaly scores to generate pseudo labels for supervised contrastive learning. SLADE [15] employs contrastive learning to mitigate the drift in dynamic node representations over short time intervals and to promote the alignment between the node representations and those generated from recent interactions.

Although these methods have achieved promising results, they still suffer from two key limitations: **1.Weak time encoding**: To integrate temporal dynamics into graph representation learning, DTDG-based methods are limited to capturing coarse-grained temporal dependencies among snapshots with classical sequence models. In contrast, CTDG-based methods typically concatenate node representations with FTE time encoding of edge timestamps to fuse the temporal information. However, such simple fusion strategy struggles to capture the complex interaction between node representations and time, failing to capture evolving dynamics. **2.Failing to capturing long-term evolutionary patterns**: To address the challenges posed by the scarcity of anomaly labels, existing methods often incorporate more generalizable self-supervision signals to enhance model's performance. Among them, methods [1,15] that leverage nodes' historical information to generate self-supervision signals have achieved superior performance. However, these methods concentrate on capturing short-term temporal dependencies by retaining and leveraging the most recent history of nodes, neglecting the capture of nodes' long-term evolutionary patterns. Intuitively, the long-term evolutionary patterns can provide a more comprehensive characterization of nodes' dynamic evolution, enabling the model to detect anomalies over extended time intervals and across multiple time points.

To address these limitations, we propose a multi-step histories-based contrastive learning framework, MHisCL, to detect anomalies in dynamic graphs in a self supervised manner. Our work is built upon an empirical yet practical observation that the representations of normal nodes tend to evolve smoothly, while those of anomalous nodes deviate significantly from their long-term histories. Specifically, we devise a history recurrent network learn, cache, and retrieve multi-step historical states for each node. Then, we utilize contrastive learning to encourage the alignment between a node's current state and its aggregated historical state, capturing the smooth long-term evolutionary patterns of normal nodes. As a result, anomalies can be efficiently detected by measuring the disagreement of the contrastive pairs through a straightforward similarity computation. Furthermore, to better discover unexpected or unusual changes in node or edge behavior compared to historical patterns, MHisCL incorporates a gated temporal graph attention network (GT-GAT), which uses a novel state space time encoding (SSTE) to capture complex evolving dynamics. Extensive experiments demonstrate the superiority of MHisCL.

Our contributions are summarized as follows:

- **Observation.** We observe that normal nodes exhibit smooth long-term evolutionary patterns, while anomalous nodes significantly deviate their long-term histories. This observation provides opportunities to identify anomalies in a self-supervised manner.
- **Multi-step histories-based contrastive learning framework.** Motivated by the observation, we propose MHisCL to capture the smooth long-term evolutionary patterns of normal nodes. Anomalies that deviate from the smooth patterns can be efficiently detected through a simple similarity computation.
- **Time encoding.** To capture complex evolving dynamics, we propose SSTE, a more efficient time encoding for temporal graph learning. SSTE employs a linear dynamical system to explicitly articulate the interactions between node representations and temporal dynamics.
- **Experimental results.** We conduct extensive experiments on several dynamic graphs. Experimental results demonstrate that MHisCL outperforms strong baselines by large margins.

2. Related work

2.1. Anomaly detection in dynamic graphs

Anomaly detection in dynamic graphs has attracted significant research attention due to its broad real-world applications. Early methods focused on using shallow mechanisms to detect anomalies. For example, GOutlier [16] employs a structural connectivity model, while CM-Sketch [17] utilizes Count-Min sketch techniques. Several methods leverage the community structure of networks for anomaly detection. CBN [18] identifies anomalous snapshots by detecting temporal changes in community boundary nodes. OBN [19] extends this approach by capturing dynamic changes in the boundary nodes of overlapping communities. CBN-O [20] identifies outlier nodes by measuring the degree to which these boundary nodes belong to their respective communities.

Recently, methods based on deep learning have become the mainstream approach for dynamic graph anomaly detection, owing to their strong ability to automatically extract complex and comprehensive information from data. Netwalk [9] utilizes a random walks-based autoencoder to learn node embeddings and then perform dynamic clustering. The anomaly score for each edge is measured by its closest distance to any cluster centers. AddGraph [10] proposed an end-to-end framework for detecting anomalous edges. It first utilizes GCN to encode structural information on each graph snapshot, and then use attention-based GRU to capture the long-term and short-term dependencies among graph snapshots. Finally, an edge anomaly detector is trained based on the node embeddings with a link prediction task. StrGNN [11] further extracts h-hop enclosing subgraph around each edge to generate edge embeddings. TADDY [2] first constructs an informative node encoding that represents the structural role and temporal status of each node, and then feeds this encoding into a transformer encoder to simultaneously capture structural and temporal information. However, these link prediction-based methods [2,10,11] suffer from noisy labels resulting from randomly negative sampling. To address this issue, RustGraph [12] generates more robust pseudo labels based on reconstructed adjacency matrices from a graph auto-encoder. FALCON [13] promotes the alignment of attention distributions between source and target nodes through contrastive learning, improving the model's generalization capabilities.

Nevertheless, most of the aforementioned methods are primarily designed for DTDGs, struggling to capture fine-grained temporal information. In contrast, SAD [1] focuses on anomaly detection in CTDGs and leverages historical predicted anomaly scores to generate pseudo labels for supervised contrastive learning, significantly improving model's performance. SLADE [15] employs contrastive learning to mitigate the drift in dynamic node representations over short time intervals and to promote the alignment between the node representations and those generated from recent interactions. Despite their success, these methods still suffer from weak time encoding and neglect the potential of capturing long-term temporal consistency for self-supervised anomaly detection in dynamic graph.Comparisons between our proposed MHisCL and existing methods are presented in Table 1.

Table 1
Comparison of MHisCL and existing methods for anomaly detection in dynamic graphs. ✓/△ indicates that the method demonstrates a strong/moderate capability in handling a certain property, while ✗ represents the method lacks this capability.

Capability	CBN [18]	OBN [19]	CBN-O [20]	Netwalk [9]	AddGraph [10]	StrGNN [11]	TADDY [2]	RustGraph [12]	FALCON [13]	SAD [1]	SLADE [15]	MHisCL
Robustness to noisy labels	✓	✓	✓	✓	△	✗	✗	△	△	△	✓	✓
Fine-grained temporal information	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓
Deep time&representation fusion	✗	✗	✗	✗	✗	✗	△	✗	△	△	△	✓
Long-term temporal consistency	△	△	✗	✗	✗	✗	✗	△	✗	✗	△	✓

2.2. Graph contrastive learning

Graph contrastive learning is a novel self-supervised learning paradigm that has gained widespread popularity in recent years due to its success in graph representation learning [21–23]. It enables learning meaningful graph representations without the need for explicit labels or supervision. The key insight behind it is to pull positive pairs with similar semantics together while pushing negative pairs with different semantics apart. DGI [22] was the first method to introduce contrastive learning into the graph domain, which aims to maximizes the mutual information between local node representations and global graph representation. GRACE [24] constructs two augmented views of the input graph and contrasts the node representations between the two views. Subsequently, many methods have been proposed to further improve performance, such as MVGRL [25], BGRL [26], and InfoGCL [23]. Apart from representation learning, contrastive learning has also been successfully applied to GAD [1,13,15,27,28], which can significantly improve the model's performance and generalization.

3. Preliminaries

In this section, we first introduce the notations of Continuous-Time Dynamic Graphs (CTDGs) and then formulate the problem of unsupervised anomaly detection in CTDGs. Finally, we present our preliminary study on the potential of capturing nodes' long-term evolutionary patterns for dynamic anomaly detection.

3.1. Continuous-time dynamic graphs (CTDGs)

We consider a continuous-time dynamic graph streaming scenario, where new observations on the dynamic graph are streamed immediately and continuously. Typically, a CTDG is constructed based on plenty of temporal events $\delta(t) = (v, u, \mathbf{e}_{vu}^t, t)$ ordered by timestamp t, each event represents an interaction occurred between source node v to target node u at timestamp t; $\mathbf{e}_{vu}^t \in \mathbb{R}^{d_e}$ is the associated edge feature, where d_e is the dimensionality. We denote by $\mathcal{E} = \{\delta(t_1), \delta(t_2), \ldots, \delta(t_m)\}$ consists of all temporal events in a CTDG where m is the number of observed events. In addition, each node v is associated with a d_x-dimensional vector $\mathbf{x}_v \in \mathbb{R}^{d_x}$ as its node feature. We denote a CTDG by $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of n nodes involved in all temporal events.

3.2. Unsupervised anomaly detection in CTDGs

Given a CTDG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, any temporal event $\delta(t) = (v, u, \mathbf{e}_{vu}^t, t) \in \mathcal{E}$ is associated with a class label $y_v^t \in \{0, 1\}$ which represents the source node v at timestamp t should be identified as either a benign node, labeled 0, or an anomalous node, labeled 1. Our goal is to learn an anomaly detection function $f(\cdot) \to [0, 1]$, which can accurately measure the node's ground-truth degree of abnormality. We address this problem in an unsupervised setting, where no labels indicating nodes' normality are used in the training phase, and the labels are only used in the test phase to evaluate the model's performance.

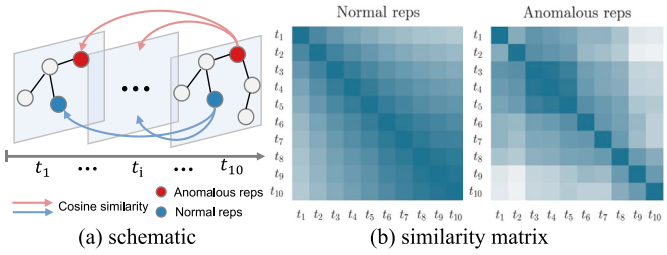


Fig. 1. Similarity among normal and anomalous representations in historical time points $\{t_1, \ldots, t_{10}\}$, respectively. For anomalous representations, t_{10} is the time point that unexpected behaviors occurred.

3.3. Preliminary investigation

In this section, we conduct preliminary experiments on the Wikipedia dataset to explore the potential of capturing nodes' long-term evolutionary patterns for self-supervised anomaly detection in dynamic graph. The dataset statistics are summarized in Table 3. We use TGAT [14] as the learning model for supervised anomaly detection and focus on the learned representations, which is obtained after converging TGAT on the dataset. For simplicity, we use anomalous/normal representations to denote the representations of anomalous/normal nodes, respectively. As our goal is to explore the underlying patterns of anomalous and normal nodes along the time dimension, we first select the last 10 anomalous/normal representations ordered by time points t_1 to t_{10} for each node. We then compute the cosine similarity $\text{sim}(\mathbf{z}_1, \mathbf{z}_2) = \frac{\mathbf{z}_1^\top \mathbf{z}_2}{\|\mathbf{z}_1\| \cdot \|\mathbf{z}_2\|}$ between all paired representations at all time points. The results are shown in Fig. 1 which illustrates the schematic of similarity computation (left) and presents the results (right). The results are averaged across all nodes, and a darker color indicates higher similarity.

According to Fig. 1, we observe that the representations of normal nodes evolve smoothly over time, exhibiting high similarity with their historical representations even over longer periods. We denote this as normal long-term evolutionary patterns. In contrast, when anomalous behavior occurs (i.e., at t_{10}), the representations of anomalous nodes remain relatively similar to their historical representations only in the short term (e.g., at t_9 and t_8) but deviate significantly from their long-term histories (e.g., from t_1 to t_7). We refer to this as anomalous long-term evolutionary patterns. The above observations reveal two distinct long-term evolutionary patterns of anomalous and normal nodes, respectively, providing a novel insight for self-supervised anomaly detection in dynamic graph, i.e., effectively distinguishing these two intrinsic patterns.

4. Method

In this section, we present the framework of MHisCL for self-supervised graph anomaly detection in dynamic graphs. The overall framework of MHisCL is presented in Fig. 2, including its architectural design, training and inference pipelines. We also summarize the overall pipeline of MHisCL in Algorithm 1.

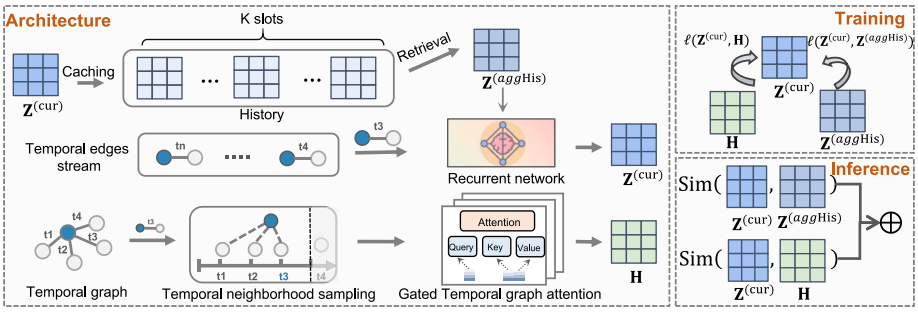


Fig. 2. Overall framework of MHisCL. MHisCL incorporates gated temporal graph attention to learn the evolving dynamics of graphs, with an additional recurrent network to handle historical states for contrastive learning.

4.1. History recurrent network

To better capture the long-term evolutionary patterns of nodes, we introduce a history recurrent network, which implicitly captures the temporal dependencies between a node's current state and its most recent multi-step historical states. Specifically, the network consists of two key components: (1) a history module, which caches and retrieves the most recent multi-step historical states for each node. (2) a recurrent network, which processes temporal events sequentially and updates the node state based on the cached historical information and the current event. Given a temporal event $\delta(t) = (v, u, \mathbf{e}_{vu}^t, t)$ at time t, the overall update process for the source node v is formalized as follows:

$$\begin{aligned} &\mathbf{z}_v^{(\text{cur})} = \text{RNNCell}_\theta(\mathbf{z}_v^{(\text{in})}, \mathbf{z}_v^{(\text{aggHis})}), \\ &\mathbf{z}_v^{(\text{aggHis})} \overset{\text{aggregate}}{\leftarrow} \text{History}(v), \\ &\mathbf{z}_v^{(\text{cur})} \overset{\text{append}}{\rightarrow} \text{History}(v) \end{aligned} \tag{1}$$

where $\mathbf{z}_v^{(\text{cur})} \in \mathbb{R}^d$ represents the currently updated state of node v, and $\mathbf{z}_v^{(\text{in})} \in \mathbb{R}^d$ is the current input corresponding to the current event. $\mathbf{z}_v^{(\text{aggHis})} \in \mathbb{R}^d$ is the aggregated historical state which effectively integrates the node's multi-step historical states stored in the history module History. The module RNNCell_θ represents a recurrent neural network. After obtaining the updated state $\mathbf{z}_v^{(\text{cur})}$, it is written back to the history module for storage, ensuring that relevant historical information can be accessed in the future. Here we omit time t for notion simplicity.

History caching. The goal of history caching is to store the most recent multi-step historical states for each node, which can effectively reflect the node's long-term evolutionary patterns. To achieve that, we implement the history module History as a length-K first-in-first-out (FIFO) queue. Specifically, we initialize a set of caching matrices as $\mathbf{M} = \{\mathbf{M}_1, \ldots, \mathbf{M}_K\}$, where $\mathbf{M}_k \in \mathbb{R}^{N \times d}$ represents the history caching matrix at time slot k, with N being the number of nodes and K being the number of time slots. When a new history arrives, we drop the oldest history and store it in the queue based on its time priority.

History retrieval. The goal of history retrieval is to aggregate the stored multi-step historical states into a single state that can effectively summarizes the node's overall historical information over a given time period. For simplicity, we compute the average of these historical states across all time slots to obtain the aggregated historical state:

$$\mathbf{z}_v^{(\text{aggHis})} = \frac{1}{K} \sum_{k=1}^{K} \mathbf{M}_k(v) \tag{2}$$

where $\mathbf{M}_k(v)$ represents the stored historical state of node v at time step k.

Recurrent neural network. Since a continuous-time dynamic graph is represented as a sequence of timed events, we utilize a recurrent neural network (RNN) to capture long-term temporal dependencies among these events and learn meaningful node states for downstream anomaly detection. In this context, the input $\mathbf{z}_v^{(\text{in})}$ to the network can be the projection of initial edge feature of each event. Furthermore, we implement RNNCell_θ with a Gated Recurrent Unit (GRU), a simple yet effective recurrent model, to process these events. Finally, the update process for the event $\delta(t) = (v, u, \mathbf{e}_{vu}^t, t)$ in Eq. (1) can be rewritten as follows:

$$\begin{aligned} &\mathbf{z}_v^{(\text{cur})}(t) = \text{GRU}(\mathbf{z}_v^{(\text{in})}(t), \mathbf{z}_v^{(\text{aggHis})}(t)), \\ &\mathbf{z}_v^{(\text{in})}(t) = \text{MLP}(\mathbf{e}_{uv}^t) \end{aligned} \tag{3}$$

where MLP is a multi-layer perceptron, and $\mathbf{z}_v^{(\text{aggHis})}(t)$ is the aggregated historical state for node v retrieved from History at time t. For notation simplicity, we omit the time t and denote $\mathbf{Z}^{(\text{cur})}$ and $\mathbf{Z}^{(\text{aggHis})}$ as the current and aggregated historical states of all nodes.

4.2. Gated temporal graph attention network

The history recurrent network primarily focuses on capturing the long-term temporal dependencies by processing timed edge events sequentially, but it overlooks the valuable temporal neighborhood information of graphs. To effectively combine structural and temporal information, we propose a Gated Temporal Graph Attention Network (GT-GAT). Specifically, GT-GAT consists of three key modules: state space time encoding (SSTE), gated temporal message and temporal graph attention.

4.2.1. State space time encoding

To effectively learn dynamic-aware representations for temporal graphs, it is crucial to leverage the time information associated with each streamed event. The prevailing practice in representing temporal information is to utilize the functional time encoding (FTE) method [14, 29], which originates from the seminal work of random Fourier features [30] that construct functional feature maps of time gaps whose induced kernel is shift-invariant. For example, the commonly used FTE in [14] is defined as follows:

$$\Phi_{\text{FTE}}(\Delta) = \text{Cos}\left(\Delta \mathbf{W}_t + \mathbf{b}_t\right), \tag{4}$$

where Δ is some relative time span and $\mathbf{W}_t$ and $\mathbf{b}_t$ are learnable parameters. The FTE encoding is often used in conjunction with temporal attention mechanisms due to the intimate connection between attention and kernel methods [31]. In this context, the FTE encoding is often concatenated with node representations as an additional temporal feature for attentive neighborhood aggregation. However, such simple time encoding scheme fails to capture the complex interactions between temporal information and node representations, such as the dynamic evolution of node representations over time.

To better capture the complex evolving dynamics, we draw insights from recent developments of state space models [32–34] and propose a more efficient time encoding, called state space time encoding (SSTE). Specifically, SSTE adopts a linear dynamical system to explicitly model the dynamic evolution of node states over time, thus effectively capturing complex evolving dynamics. Formally, given a node u that emerges

at time t_u with its intermediate node representation $\mathbf{h}_u(t_u)$ at t_u (defined in the following Section 4.2.3), and a termination time t_v, the evolving process of its state during the time span $t_v - t_u$ can be defined as follows:

$$\frac{d\mathbf{s}_u(t)}{dt} = \mathbf{A}\mathbf{s}_u(t) + \mathbf{B}\mathbf{h}_u(t), \ t \in [t_u, t_v] \tag{5}$$

where the matrices $\mathbf{A} \in \mathbb{R}^{d\times d}, \mathbf{B} \in \mathbb{R}^{d\times d}$ are learnable parameters of the system, and $\mathbf{s}_u(t) \in \mathbb{R}^d$, $\mathbf{h}_u(t) \in \mathbb{R}^d$ are the state and input of node u at time t, respectively. Since the node features are observed only at its emergence, we set $\mathbf{h}_u(t) \equiv \mathbf{h}_u(t_u) = \mathbf{h}_u$ to be a constant input signal. The system (5) is a multi-input, multi-output (MIMO) state space model (SSM), and the differential equation (5) has a closed form solution, with its state at termination time t_v expressed as:

$$\begin{aligned}\mathbf{s}_u(t_v) &= e^{\mathbf{A}(t_v-t_u)}\mathbf{s}_u(t_u) + \mathbf{A}^{-1}\left(e^{\mathbf{A}(t_v-t_u)} - I\right)\mathbf{B}\mathbf{h}_u \\ &\approx e^{\mathbf{A}(t_v-t_u)}\mathbf{s}_u(t_u) + (t_v - t_u)\mathbf{B}\mathbf{h}_u\end{aligned} \tag{6}$$

Following recent practices [35], we restrict $\mathbf{A}$ to be a diagonal matrix with non-positive diagonals. The resulting solution (6) comprises two components: The first term implies a diminishing effect of the initial state $\mathbf{s}_u(t_u)$, while the second term indicates an accumulated effect of $\mathbf{h}_u$ that grows with the duration $t_v - t_u$. Combining these two components allows the model to effectively capture the long-term evolutionary dynamics. To further improve the expressive power of the system, we set the initial state as an affine transformation of the input and term the resulting state as *state space time encoding* (SSTE) of temporal information:

$$\Phi_{\text{SSTE}}(\Delta, \mathbf{h}) = e^{\mathbf{A}\Delta}\left(\mathbf{W}_S\mathbf{h} + \mathbf{b}_S\right) + \Delta\mathbf{B}\mathbf{h}. \tag{7}$$

Here we use Δ to denote some time span between the termination time t_v and the starting time t_u of the evolution, and $\mathbf{W}_S \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_S \in \mathbb{R}^d$ are learnable parameters.

4.2.2. Gated temporal message

Classical graph neural networks (GNNs) utilize a message-passing mechanism to capture intricate relationships within graph structures, i.e., aggregating static neighborhood messages. To extend GNNs to dynamic graphs, we fuse the obtained time encodings with intermediate node representations to form temporal messages. Inspired by the gated multi-layer perceptron (MLP) architecture, we employ a gating mechanism for this fusion. The resulting gated temporal message is computed as follows:

$$\text{TM}(\Delta, \mathbf{h}) = \sigma\left(\mathbf{C}\Phi_{\text{SSTE}}(\Delta, \mathbf{h})\right) \odot (\mathbf{D}\mathbf{h}), \tag{8}$$

where σ is the SeLU nonlinearity [36], and $\mathbf{C}, \mathbf{D} \in \mathbb{R}^{d\times d}$ are projection parameters. Compared to simple concatenation, the temporal gate can selectively pass or block information flow within the neighborhood, thereby enabling more sophisticated and adaptive representations.

4.2.3. Temporal graph attention

After obtaining the temporal messages, we leverage self-attention mechanism [37] to aggregate these messages from neighboring nodes with different temporal importance, thus effectively combining structural and temporal information. Specifically, given any temporal event involving a source node v at time t_v, we aim to produces its time-aware node representation through temporal neighborhood aggregation. To achieve this, we first sample its temporal neighborhood $\mathcal{N}(v; t_v) = \{u_1, \ldots, u_N\}$, where u_j denotes the adjacent node of v with an observation $(v, u, \mathbf{e}_{vu}^{t_u}, t_u)$ occurred prior to t_v. Then, we compute its node representation by stacked temporal graph attention layers:

$$\begin{aligned}
\mathbf{Q}_v^{(l)} &= \mathbf{W}_Q^{(l)}\mathbf{h}_v^{(l-1)}(t_v), \\
\mathbf{K}_u^{(l)} &= \text{TM}_K(t_v - t_u, \mathbf{e}_{vu}^{t_u} \parallel \mathbf{h}_u^{(l-1)}(t_u)), \\
\mathbf{V}_u^{(l)} &= \text{TM}_V(t_v - t_u, \mathbf{e}_{vu}^{t_u} \parallel \mathbf{h}_u^{(l-1)}(t_u)), \\
\tilde{\mathbf{h}}_v^{(l)}(t_v) &= \sum_{u\in\mathcal{N}(v;t_v)} \frac{e^{\frac{1}{\sqrt{d_K}}\left\langle \mathbf{Q}_v^{(l)}, \mathbf{K}_u^{(l)}\right\rangle}}{\sum_{u\in\mathcal{N}(v;t_v)} e^{\frac{1}{\sqrt{d_K}}\left\langle \mathbf{Q}_v^{(l)}, \mathbf{K}_u^{(l)}\right\rangle}} \mathbf{V}_u^{(l)}, \\
\mathbf{h}_v^{(l)}(t_v) &= \text{MLP}^l(\tilde{\mathbf{h}}_v^{(l)}(t_v) \parallel \mathbf{h}_v^{(l-1)}(t_v)),
\end{aligned} \tag{9}$$

where $\parallel$ denotes the concatenation operation, $\mathbf{h}_v^{(l)}(t_v) \in \mathbb{R}^d$, $\tilde{\mathbf{h}}_v^{(l)}(t_v) \in \mathbb{R}^d$ are the hidden representation and aggregated neighborhood representation of node v at layer l and time t_v. Initially, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$. The matrices $\mathbf{W}_Q^{(l)} \in \mathbb{R}^{d_Q\times d}$ is the weight matrices of 'query' in self-attentions, and the two separate temporal messages TM_K and TM_V represent the key and value encodings, respectively. Here we set the termination time in SSTE to t_v for each neighboring node u, aiming to encode the cumulative influence of node u over v during the timespan $t_v - t_u$. For simplicity, we denote the final representation of node v in the last layer L as $\mathbf{h}_v(t_v)$. Further, we omit time t_v and refer to $\mathbf{H}$ as the node representations.

4.3. Dual contrastive learning

After obtaining the node states $\mathbf{Z}^{(cur)}$, $\mathbf{Z}^{(aggHis)}$ and node representations $\mathbf{H}$, we propose a dual contrastive learning framework for self-supervised anomaly detection. This framework consists of two components: Temporal Consistent Contrast (TCC) and Temporal Structural Contrast (TSC).

4.3.1. Temporal consistent contrast

Our preliminary study in 3.3 reveal that the representations of normal nodes tend to evolve smoothly over time, while those of anomalous nodes exhibit significant deviations from their long-term histories. This observation provides a novel self-supervised approach for anomaly detection, i.e., effectively identifying the intrinsic evolutionary patterns. Since the majority of nodes in the training set are normal, we introduce a multi-step histories-based contrastive learning, termed Temporal Consistent Contrast (TCC), to capture the smooth long-term evolutionary patterns of normal nodes. Specifically, TCC encourages the alignment of each node's current state with its aggregated multi-step historical states, thereby effectively capturing long-term temporal consistency. Formally, for each batch B with temporal edges $\mathcal{E}_B$ and corresponding node set $\mathcal{V}_B$, the temporal consistent contrastive loss can be defined as follows:

$$\mathcal{L}_{\text{tcc}} = \frac{1}{|\mathcal{E}_B|}\sum_{(v,u,\mathbf{e}_{uv}^t,t)\in\mathcal{E}_B} -\log\frac{e^{\text{sim}\left(\mathbf{z}_v^{(\text{cur})}(t), \mathbf{z}_v^{(\text{aggHis})}(t)\right)/\tau}}{\sum_{v'\in\mathcal{V}_B} e^{\text{sim}\left(\mathbf{z}_v^{(\text{cur})}(t), \mathbf{z}_{v'}^{(\text{aggHis})}(t)\right)/\tau}}, \tag{10}$$

where τ is the temperature hyperparameter and $\text{sim}(\cdot,\cdot)$ is the similarity measure such as cosine similarity. The aggregated historical state $\mathbf{z}_{v'}^{(\text{aggHis})}(t)$ of other nodes serves as negative samples, promoting the model's ability to differentiate between normal and anomalous evolutionary patterns.

4.3.2. Temporal structural contrast

In addition to temporal consistency, structural information is also crucial for anomaly detection. It is expected that anomalous and normal nodes would exhibit distinct local contexts. While the node state $\mathbf{Z}^{(cur)}$ primarily captures long-term temporal dependencies, the aggregated node representation $\mathbf{H}$ encodes informative temporal neighborhood information. Therefore, we introduce Temporal Structural Contrast (TSC) to capture the temporal structural consistency of normal nodes. Specifically, TSC treats the current node state $\mathbf{z}_v^{cur}(t)$ and the aggregated node representation $\mathbf{h}_v(t)$ as two augmented views of the same node v, and encourage the agreement between them. Formally, the temporal structural contrastive loss is defined as follows:

$$\mathcal{L}_{\text{tsc}} = \frac{1}{|\mathcal{E}_B|}\sum_{(v,u,\mathbf{e}_{uv}^t,t)\in\mathcal{E}_B} -\log\frac{e^{\text{sim}\left(\mathbf{h}_v(t), \mathbf{z}_v^{(\text{cur})}(t)\right)/\tau}}{\sum_{v'\in\mathcal{V}_B} e^{\text{sim}\left(\mathbf{h}_v(t), \mathbf{z}_{v'}^{(\text{cur})}(t)\right)/\tau}}, \tag{11}$$

4.3.3. Objective

Combining these two contrastive losses, the final training objective can be formalized as:

$$\mathcal{L} = \mathcal{L}_{\text{tcc}} + \alpha\mathcal{L}_{\text{tsc}} \tag{12}$$

where weight α is a hyperparameter that controls the contribution of temporal structural contrastive loss.

Table 2
Time and space complexity comparisons of different models. T represents the number of snapshots, $\tilde{n}$ represents the average number of nodes in each snapshot, ω represents the snapshot window size and k represents the number of sampled neighbors for each node.

Complexity	AddGraph [10]	TADDY [2]	JODIE [38]	TGAT [14]	TGN [39]	SAD [1]	SLADE [15]	MHisCL
Time	$O(Lmd^2 + Tn\omega d^2)$	$O(m\omega kd^2 + T\tilde{n}^2)$	$O(md^2)$	$O(mk^L d^2)$	$O(mk^L d^2)$	$O(mk^L d^2)$	$O(mkd^2)$	$O(mk^L d^2 + mKd)$
Space	$O(Ld^2 + \omega nd)$	$O(Ld^2 + T\tilde{n}^2)$	$O(d^2 + nd)$	$O(Ld^2)$	$O(Ld^2 + nd)$	$O(Ld^2)$	$O(Ld^2 + nd)$	$O(Ld^2 + Knd)$

4.4. Anomaly scoring

The dual contrastive learning framework is designed to capture the normal temporal and structural patterns of nodes. In this context, the similarity between a node's positive contrastive pair (e.g., $\mathbf{z}_v^{(\text{cur})}(t)$ and $\mathbf{z}_v^{(\text{aggHis})}(t)$) indicates how well the node conforms to the normal patterns. Therefore, once the model is trained, we can efficiently identify anomalous nodes by detecting those whose positive contrastive pairs exhibit relatively low similarity. Specifically, given an edge event $\delta(t) = (v, u, \mathbf{e}_{vu}^t, t)$, we can measure the anomaly score of the source node v at time t as follows:

$$\rho_v = \frac{1}{4}\left(2 - \text{sim}(\mathbf{z}_v^{(\text{cur})}(t), \mathbf{z}_v^{(\text{aggHis})}(t)) - \text{sim}(\mathbf{z}_v^{(\text{cur})}(t), \mathbf{h}_v(t))\right) \tag{13}$$

A higher value of ρ_v indicates a higher probability that the node v is an anomaly. This simple similarity computation provides a more efficient approach for anomaly scoring compared to training an additional classifier.

4.5. Complexity analysis

The proposed MHisCL consists of two major modules: the history recurrent network and the GT-GAT. To assess the computational complexity, we first analyze each module separately and then compute the overall complexity. For simplicity, we assume that the dimension of both input features and hidden representations is d.

First, we consider the time complexity. When a new edge arrives, the aggregated historical state for the source node is computed by averaging its cached historical states across K time slots, which requires $O(Kd)$. The GRU then generates a updated state for the node by incorporating both the aggregated historical state and the current edge, which takes $O(d^2)$. Therefore, the total time complexity of the recurrent network is $O(mKd + md^2)$, where m is the number of edges. For the GT-GAT, at each layer, k neighbors are sampled for each edge to compute attentions. Consequently, the time complexity for the L-layer GT-GAT is $O(mk^L d^2)$. Combining both modules, the overall time complexity of MHisCL is $O(mk^L d^2 + mKd)$.

Next, we analyze the space complexity. The history recurrent network requires $O(d^2)$ parameters for the GRU. Additionally, it needs to cache the most recent K historical states for each node, introducing an extra space overhead of $O(Knd)$, where n is the number of nodes. For the L-layer GT-GAT, it has $O(Ld^2)$ parameters. Thus, the total space complexity of MHisCL is $O(Ld^2 + Knd)$.

Finally, we compare MHisCL with seven classical methods in terms of the complexity. As shown in Table 2, DTDG-based methods (e.g., AddGraph and TADDY) exhibit the highest complexity due to the additional time and space required for preprocessing each snapshot and handling the temporal dependencies between consecutive snapshots. In contrast, CTDG-based methods process each edge independently, whose time complexity scales linearly with the number of edges. Furthermore, since all CTDG-based models are built upon TGAT, RNNs, or combinations of both, their time complexities are all approximately $O(mk^L d^2)$. However, aiming to capture smooth long-term evolutionary patterns, MHisCL requires caching multi-step historical states for each node, which results in a linearly increased time and space complexity with respect to the number of time slots K. In practice, since $K \ll d$, the overall complexity of MHisCL remains highly comparable to that of the baseline models.

Algorithm 1: Algorithm of MHisCL

Input: Dynamic graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; Training edges $\mathcal{E}_{train}$; Test edges $\mathcal{E}_{test}$; Number of training epochs T; Batch size B;
Output: Node anomaly scoring function $f(\cdot)$;

Randomly initialize the trainable parameters Θ for history recurrent network and Φ for gated temporal graph attention network;
// Training stage
for $t = 1, 2, \ldots, T$ **do**
  $\mathcal{E}_B \leftarrow$ Split $\mathcal{E}_{train}$ chronologically into batches of size B;
  for *batch* $\mathcal{E}_b = (\delta(t_1), \ldots, \delta(t_B)) \in \mathcal{E}_B$ **do**
    Calculate nodes' current states $\mathbf{Z}^{(cur)}$ via Eq. (3), and aggregated historical states $\mathbf{Z}^{(aggHis)}$ via Eq. (2);
    Calculate node representations $\mathbf{H}$ via Eq. (9);
    Compute the loss objective via Eq. (10), (11) and (12);
    Backpropagate and update the parameters Θ and Φ with gradients.
  end
end
// Inference Stage
for $\delta(t) = (v, u, \mathbf{e}_{vu}^t, t) \in \mathcal{E}_{test}$ **do**
  Calculate the source node's anomaly score $\rho_v = f(\delta(t))$ via Eq. (13);
end

5. Experiments

In this section, we conduct extensive experiments on various datasets to validate the effectiveness of MHisCL. Specifically, we aim to explore the following questions: **RQ1.** How does MHisCL perform compared to other state-of-the-art dynamic graph anomaly detection methods? **RQ2.** How does each component in MHisCL impact the overall performance? **RQ3.** How does different hyperparameters settings influence the performance of MHisCL? **RQ4.** How does MHisCL perform with limited training data compared to other state-of-the-art models? **RQ5.** How effective is the model architecture of MHisCL compared to other alternatives?

5.1. Datasets

We evaluate the performance of MHisCL on six dynamic graph datasets, including three public interaction datasets Wikipedia, Reddit, Mooc [38], two transaction graphs Bitcoin-Alpha and Bitcoin-OTC [40], as well as an user-product rating network [41]. The dataset statistics are listed in Table 3, and below are the detailed descriptions of these datasets:

- **Wikipedia** [38] records the edits made by users on Wikipedia pages over time. Each edit is represented as an interaction, where a user (source node) modifies a page (target node). The dataset includes dynamic labels that mark users who are banned after a specific edit as anomalous.
- **Reddit** [38] collects posts made by users on subreddits. Each post is an interaction between a user (source node) and a subreddit (target node). Similar to the Wikipedia dataset, the dynamic labels represents banned users from Reddit.

Table 3
Statistics of six dynamic graph anomaly detection datasets

Dataset	#Nodes	#Edges	#Snapshots	#Average nodes per snapshot	#Average edges per snapshot	%Anomalies
Wikipedia	9227	157,474	152,757	2.06	1.03	0.138%
Reddit	10,984	672,447	588,915	2.28	1.14	0.054%
Mooc	7074	411,749	345,600	2.33	1.19	0.987%
Bitcoin-Alpha	3783	24,186	1647	15.70	14.68	3.614%
Bitcoin-OTC	5881	35,592	35,427	2.00	1.00	7.215%
Amazon	330,317	568,454	3168	273.76	179.43	13.504%

- **Mooc** [38] consists of actions performed by students in a MOOC online course. Each action is an interaction between a student (source node) and a course element (target node). Students who drop out of the course are labeled as anomalies.
- **Bitcoin-Alpha and Bitcoin-OTC** [40] are two rating networks collected from two distinct Bitcoin trading platforms, Alpha and OTC. In these datasets, nodes represent individual users, and an edge is created when one user rates another. These ratings are used to identify anomalous users.
- **Amazon** [41] is a user-product review graph, where each interaction represents a review of a product (target node) made by a user (source node). Additionally, each review/edge is associated with some votes indicating its helpfulness. Following [42], reviews/edges with at least 10 votes are normal if the fraction of helpful-to-total votes is $\geq$ 0.75, and anomalies if $\leq$ 0.25. The other left reviews/edges are labeled as unknown, which are not involved in the evaluation process. The dataset consists of 330,318 nodes, 568,454 edges, 17,140 labeled normal edges, and 2676 labeled anomalous edges.

5.2. Baselines

We compare our proposed methods with a wide range of baselines, including (1) static methods: Radar [5], DOMINANT [4], GDN [7], SemiGNN [6]; (2) temporal methods: AddGraph [10], TADDY [2], JODIE [38], TGAT [14], TGN [39], SAD [1], SLADE [15]. The details about them are as follows:

Radar [5] detects anomalies by analyzing the structural properties of the graph. **DOMINANT** [4] is a autoencoder-based approach which reconstructs the adjacency matrix and node attributes to identify anomalies. **GDN** [7] utilizes a graph deviation network to detect anomalies with few shot labeled anomalies. **SemiGNN** [6] leverages both labeled and unlabeled data to enhance anomaly detection with a hierarchical attention mechanism.

AddGraph [10], TADDY [2] are two DTDG based anomaly detection methods. **AddGraph** [10] uses GCN and GRU to capture structural information and temporal information, respectively. **TADDY** [2] utilizes a transformer with temporal-spatial node encodings to capture structural information and temporal information simultaneously.

JODIE [38], TGAT [14], TGN [39] are three popular CTDG modeling methods. **JODIE** [38] utilizes recurrent neural networks (RNNs) to capture the temporal evolution of node representations. **TGAT** [14] introduces functional time encoding (FTE) and combine it with self-attention mechanism. **TGN** [39] combines both approaches. These three methods first train an encoder through link prediction-based self-supervised learning, and then finetune a classifier with labels. **SAD** [1] leverages the majority of unlabeled data to improve the detection performance. It utilizes graph deviation network to generate pseudo labels for supervised contrastive learning. **SLADE** [15] mitigates the drift in dynamic node representations over short time intervals and promotes the alignment between the node representations and those generated from recent interactions to detect anomalies in an unsupervised manner.

5.3. Experiment setup

For dataset split, we follow the transductive setting used in previous work [1,14,38], where datasets are split chronologically to simulate the real situations. Specifically, we train all models on the first 70% interaction events, validate on the next 15% and test on the last remaining events. For evaluation metric, We use the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) to quantify the model's performance. A higher AUC value indicates better anomaly detection performance. We conducted all experiments 10 times on NVIDIA RTX 4090 GPU with 24 GB memory, and report the average results along with their standard deviations.

5.4. Parameter settings

In this experiment, we perform a grid search to obtain the optimal hyperparameter settings for each dataset. Specifically, based on empirical findings and preliminary experiments, we set the search ranges as follows: the number of time slots K to $[1, 3, 5, 7, 10, 14, 20]$, the embedding dimension d to $[64, 128]$, temperature τ to $[0.001, 0.002, 0.005, 0.1, 0.2]$, loss weight α for $\mathcal{L}_{\text{tsc}}$ to $[0.5, 1, 2]$, learning rate to $[0.0005, 0.0001]$, and training epochs to $[5, 10, 20, 30, 35]$. Finally, we use a two-layer GT-GAT with an embedding dimension of $d = 64$ for Wikipedia, Mooc and Amazon, and $d = 128$ for Reddit, Bitcoin-OTC, and Bitcoin-Alpha. For all datasets, temperature τ and loss weight α are set to 0.02 and 1, respectively. The number of time slots K is set to 14 for Wikipedia, 5 for Reddit, 3 for Mooc, 20 for Bitcoin-Alpha and Amazon, and 10 for Bitcoin-OTC. The framework is optimized using the Adam optimizer with a learning rate of 0.0005 for all datasets, except for Reddit and Mooc, which use a learning rate of 0.0001. The training epochs are set to 30 for Wikipedia, 20 for Reddit and Bitcoin-Alpha, 35 for Mooc, 5 for Bitcoin-OTC and 10 for Amazon. Moreover, following previous work [14,39], we adopt batch processing for training MHisCL. Specifically, rather than processing the entire graph with all edges simultaneously, the temporal edge stream is chronologically divided into fixed-size batches, with each batch being processed sequentially. The batch size is fixed at 256 for all datasets. For each edge in a batch, we sample 15 one-hop neighbors and 10 two-hop neighbors to construct a subgraph for GT-GAT, except for the Reddit dataset, where 30 one-hop and 15 two-hop neighbors are sampled. By combining batch processing with subgraph sampling, MHisCL scales efficiently to large-scale datasets. The overall hyperparameter settings are summarized in Table 4. Our code is available at https://github.com/Yun-Fu/MHisCL.git.

6. Experiment results

6.1. Overall performance

To answer **RQ1**, we compare our proposed MHisCL with eleven baseline methods. The comparison results are presented in Table 5 and illustrated in Fig. 3. According to the results, we summarize the following observations:

First, temporal methods generally achieve better performance than static methods (methods designed for static graphs). This observation highlights the significant difference between anomaly detection on dynamic graphs and static graphs, emphasizing the importance of

Table 4
Hyperparameter settings for different datasets.

	Wikipedia	Reddit	Mooc	Bitcoin-Alpha	Bitcoin-OTC	Amazon
Time slots K	14	5	3	20	10	20
Embedding dimension d	64	128	64	128	128	64
Temperature τ	0.02	0.02	0.02	0.02	0.02	0.02
Loss weight α	1	1	1	1	1	1
Batch size	256	256	256	256	256	256
Number k of sampled neighbors	[15, 10]	[30, 15]	[15, 10]	[15, 10]	[15, 10]	[15, 10]
Learning rate	0.0005	0.0001	0.0001	0.0005	0.0005	0.0005
Epochs	30	20	35	20	5	10

Table 5
Overall performance of all methods in terms of AUC score (%). The best results are highlighted in **boldface**, and the second best results are highlighted with underline. "OOM" denotes that the model runs out of memory.

Method	Wikipedia	Reddit	Mooc	Bitcoin-Alpha	Bitcoin-OTC	Amazon
Radar [5]	82.91 ± 0.97	61.46 ± 1.27	62.14 ± 0.89	65.46 ± 0.72	69.37 ± 0.43	OOM
DOMINANT [4]	85.84 ± 0.63	64.66 ± 1.29	65.41 ± 0.72	67.66 ± 0.83	70.12 ± 0.91	OOM
GDN [7]	85.12 ± 0.69	67.02 ± 0.51	66.21 ± 0.74	69.02 ± 0.88	71.48 ± 0.62	OOM
SemiGNN [6]	84.65 ± 0.82	64.18 ± 0.78	64.98 ± 0.63	66.41 ± 0.75	70.55 ± 0.37	OOM
AddGraph [10]	83.91 ± 0.88	64.66 ± 0.54	66.37 ± 0.82	70.22 ± 0.89	69.91 ± 1.03	OOM
TADDY [2]	84.72 ± 1.01	67.95 ± 0.94	68.47 ± 0.76	69.96 ± 1.36	68.67 ± 0.98	OOM
JODIE [38]	85.75 ± 0.17	61.47 ± 0.58	67.91 ± 0.52	73.53 ± 1.26	69.13 ± 0.96	59.50 ± 1.41
TGAT [14]	83.24 ± 1.11	65.12 ± 1.65	66.88 ± 0.68	71.67 ± 0.97	68.33 ± 1.23	55.43 ± 2.21
TGN [39]	88.37 ± 0.35	67.16 ± 1.33	67.07 ± 0.73	71.73 ± 0.98	76.23 ± 0.23	67.57 ± 1.10
SAD [1]	86.15 ± 0.63	68.45 ± 1.27	69.44 ± 0.87	75.09 ± 1.18	66.16 ± 4.35	51.08 ± 1.33
SLADE [15]	88.68 ± 0.39	**72.19 ± 0.60**	71.02 ± 0.25	76.92 ± 0.36	77.18 ± 0.27	65.61 ± 2.28
MHisCL	**90.85 ± 1.15** **+2.17%**	71.22 ± 1.98 −0.97%	**74.78 ± 0.84** **+3.76%**	**80.78 ± 1.39** **+3.86%**	**79.70 ± 0.94** **+2.52%**	**68.40 ± 1.23** **+0.87%**

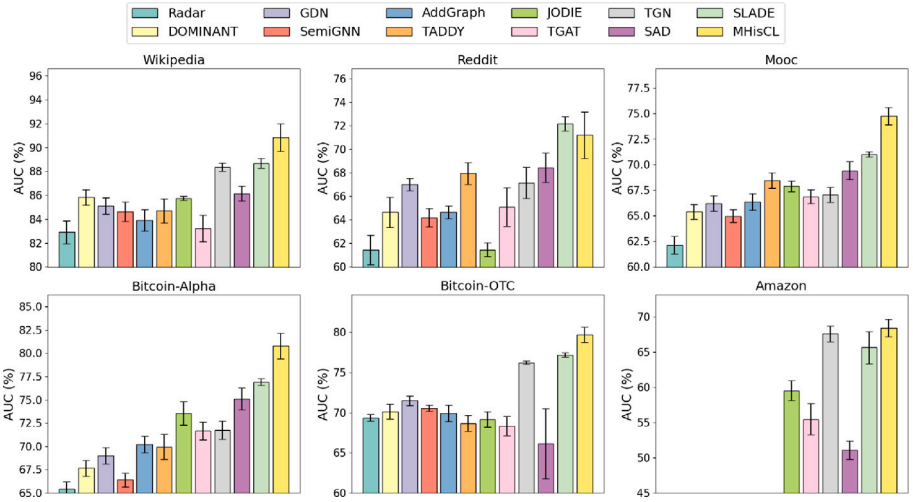


Fig. 3. AUC values of all methods on six datasets.

temporal information for dynamic graph anomaly detection. **Second**, CTDG-based methods consistently outperform DTDG-based methods (e.g., AddGraph [10], TADDY [2]) across all datasets. This can be attributed to their ability to capture fine-grained temporal information associated with edge streams. In contrast, DTDG-based methods are limited to capturing coarse-grained temporal dependencies among the snapshots, struggling to detect anomalies arising from subtle temporal changes. Moreover, due to the substantial memory overhead of the large adjacency matrix, both static-based and DTDG-based methods suffer from out-of-memory (OOM) problem when applied to the large-scale Amazon dataset with over 300K nodes. In contrast, CTDG-based models are trained on chronologically divided batches of edges, enabling them to scale efficiently to large datasets. **Third**, temporal methods (e.g., SAD [1], SLADE [15], and MHisCL) that incorporate self-supervised learning significantly outperform semi-supervised methods that rely solely on labeled data (e.g., JODIE [38], TGAT [14], TGN [39]). This demonstrates the vast potential of leveraging unlabeled data to improve model performance. Self-supervised learning enables models to learn more generalizable representations, which is particularly beneficial in the context of highly imbalanced datasets with few anomalies, as shown in Table 3. **Fourth**, our proposed MHisCL that utilizes nodes' multi-step histories as self-supervision signals outperforms all baseline models by large margins on five out of six datasets, achieving the second-best performance on the Reddit dataset. Specifically, compared to the previous state-of-the-art method, MHisCL achieves a 2.17% performance improvement on Wikipedia, a 3.76% improvement on Mooc, a 3.86% improvement on Bitcoin-Alpha, a 2.52% improvement on Bitcoin-OTC and a 0.87% improvement on Amazon. This results underscore the superiority of capturing long-term temporal consistency for self-supervised dynamic anomaly detection.

6.2. Efficiency

To evaluate the efficiency of MHisCL, we compared the training time of each epoch and test time for different models on the Wikipedia,

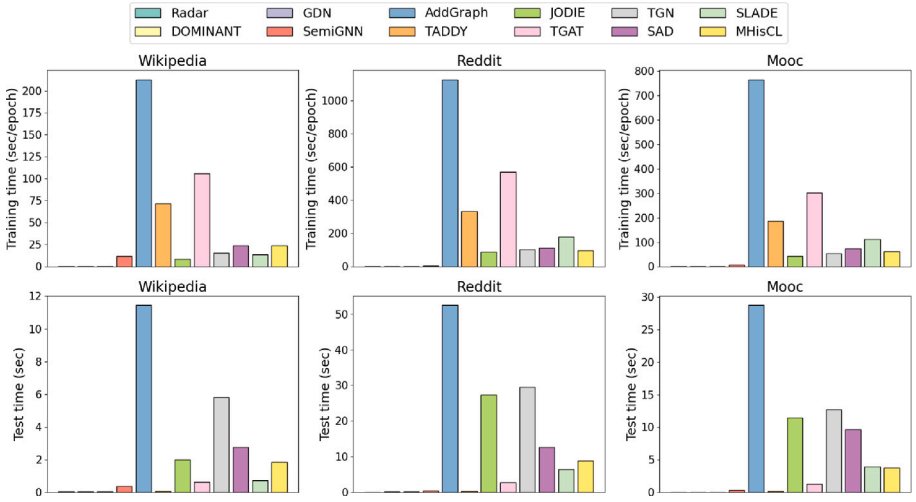


Fig. 4. Training time of each epoch and test time for different models on the Wikipedia, Reddit and Mooc datasets.

Reddit and Mooc datasets. The comparison results are shown in Fig. 4. According to these results, we summarize the following observations:

First, static graph-based methods exhibit negligible runtime, as they discard temporal information and only train the model on the static graph at the final timestamp. In contrast, temporal models are trained on multiple chronologically divided snapshots or batches of edges, which requires more training rounds for update. **Second**, the training time of DTDG-based methods is significantly higher than that of CTDG-based methods. This increased runtime can be attributed to the computationally expensive link prediction-based loss, which requires scoring numerous negative edges during training. For instance, to avoid sampling false negatives, AddGraph [10] repeatedly samples negative edges until finding one with a higher anomaly score than the current positive edge. On the other hand, TADDY [10] must re-extract contextual nodes from consecutive snapshots and recompute both structural and temporal encodings for each negative edge. **Third**, among CTDG-based methods, JODIE [38], which is built solely on RNNs, demonstrates the shortest training time. In contrast, TGN [39] and SLADE [15], which combine RNNs with a single layer of TGAT [14], show slightly higher latency. However, using a two-layer TGAT results in a significant increase in training time. This can be attributed to the exponential expansion of sampled neighbors, resulting in a more computationally expensive process of scoring negative edges. In contrast, SAD [1] eliminates the need for negative sampling by directly training TGAT with labels, thus reducing the overall training time. Similarly, MHisCL, which uses node-level contrastive loss for training, demonstrates highly comparable efficiency to other CTDG-based methods. **Fourth**, we observe that JODIE and TGN, which exhibit shorter training time, demonstrate higher test time compared other CTDG-based models. This is because these models continue to compute the representations of negative sampling nodes during testing, which introduces additional computational overhead. **Finally**, MHisCL, which combines both RNNs and GNNs to achieve superior performance, exhibits slightly longer runtime compared to models using a single architecture. Compared to TGN, which adopts a similar model architecture (i.e., combining both RNNs and GNNs) but includes a supervised classifier, our self-supervised MHisCL demonstrates nearly identical training time. However, unlike TGN, MHisCL identifies anomalies through simple similarity computations, without requiring an additional classifier. This simpler anomaly detection mechanism results in a 3× evaluation speedup over TGN.

6.3. Ablation study

To answer **RQ2**, we conduct an ablation study to verify the effectiveness of each component in MHisCL. We compare MHisCL wit its

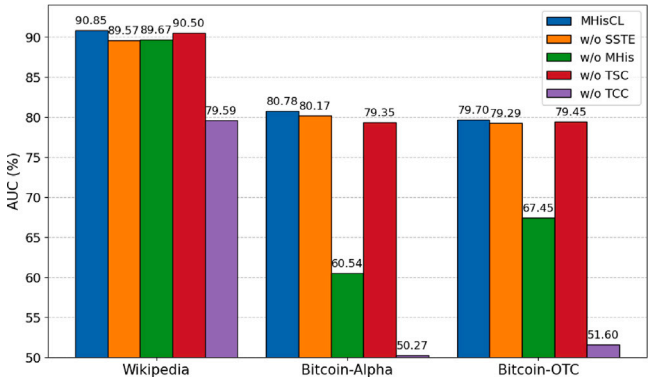
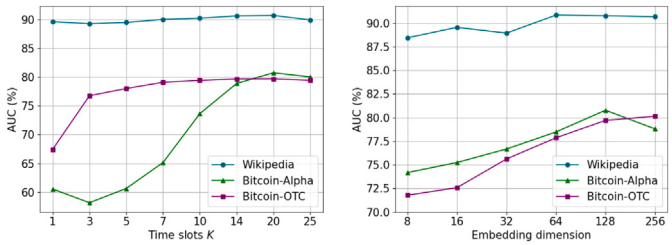


Fig. 5. Ablation study results on the Wikipedia, Bitcoin-Alpha, and Bitcoin-OTC datasets.

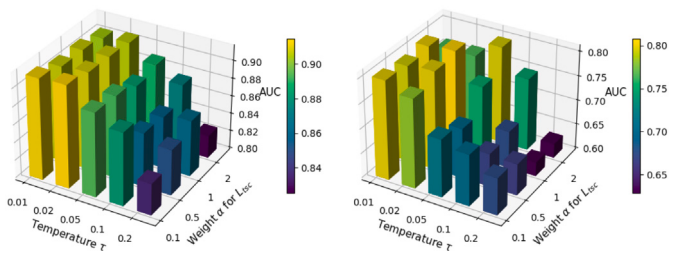
four variants: **w/o SSTE**, **w/o MHis**, **w/o TCC** and **w/o TSC**. Among them, **w/o SSTE** replaces the proposed gated temporal graph attention network incorporating SSTE with the standard TGAT [14], where hidden representations are directly concatenated with FTE to compute attentions. **w/o MHis** removes multi-step historical information, using a single-slot queue to retain only the most recent historical state for each node. **w/o TCC** removes the temporal consistent contrastive loss $\mathcal{L}_{\text{tcc}}$ and its associated similarity computations in anomaly scoring. Similarly, **w/o TSC** removes the temporal structural contrastive loss $\mathcal{L}_{\text{tsc}}$. We conduct the ablation study on the Wikipedia, Bitcoin-Alpha, and Bitcoin-OTC datasets, and the ablation results are presented in Fig. 5. Based on the results, we can conclude that:

Firstly, MHisCL consistently outperforms its four variants across all datasets, demonstrating the effectiveness of each component. **Secondly**, removing multi-step historical information and retaining only the latest historical state for contrastive learning leads to significant performance drops, e.g., 20.24% and 12.25% performance degradation on the Bitcoin-Alpha and Bitcoin-OTC datasets, respectively. This underscores the importance of capturing long-term evolutionary patterns of nodes for dynamic graph anomaly detection. **Thirdly**, MHisCL with SSTE consistently outperforms its variant with FTE, achieving substantial performance gains of 1.28% and 0.61% on the Wikipedia and Bitcoin-Alpha datasets, respectively. This highlights the importance of explicitly modeling the temporal evolution of node representations to capture complex evolutionary patterns. **Fourthly**, the temporal consistent contrast (TCC) significantly improve the performance. Specifically, removing the loss $\mathcal{L}_{\text{tcc}}$ results in notable performance drops of 11.26%,



(a) Time slots versus AUC (b) Embedding dimension versus AUC

Fig. 6. Sensitivity analysis results for the number of time slots T and embedding dimension d.



(a) Wikipedia (b) Bitcoin-Alpha

Fig. 7. AUC values of MHisCL on the Wikipedia and Bitcoin-Alpha datasets with different temperature τ and loss weight α for $\mathcal{L}_{\text{tsc}}$. A yellow color indicates a higher AUC value.

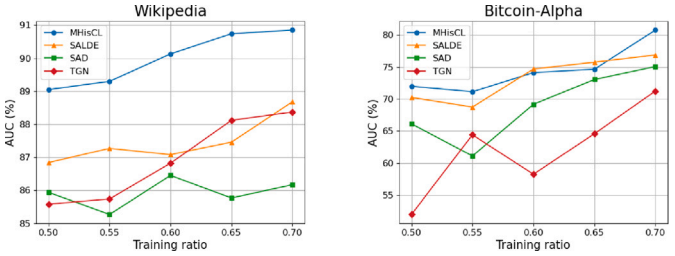


Fig. 8. The AUC values on the Wikipedia and Bitcoin-Alpha datasets when varying training ratio.

30.51%, and 28.1% on the Wikipedia, Bitcoin-Alpha, and Bitcoin-OTC datasets, respectively, emphasizing the importance of capturing temporal consistency for dynamic anomaly detection. Furthermore, combining TSC with the temporal structural contrast (TCC) yields better results across all datasets, indicating the necessity of capturing both temporal and structural patterns for effective anomaly detection in dynamic graphs.

6.4. Sensitivity analysis

To answer **RQ3**, we conduct extensive experiments to investigate the sensitivity of MHisCL to different hyperparameters, including the number of time slots K, embedding dimension d, temperature τ and weight α for $\mathcal{L}_{\text{tsc}}$.

6.4.1. Time slots

In this experiment, we investigate the impact of the number of cached historical states by varying the number of time slots K within the range {1, 3, 5, 7, 10, 14, 20, 25}. As shown in Fig. 6(a), the AUC values are relatively low at $K = 1$, indicating that capturing short-term evolutionary patterns is insufficient to effectively distinguish between normal and anomalous nodes. As K increases, the AUC values significantly increase and remain stable when $K \geq 14$, emphasizing the importance of capturing long-term evolutionary patterns for dynamic anomaly detection. However, further increasing K does not yield substantial performance gains, and a slight decline is observed at $K = 25$. This may be due to that a larger K introduces excessive distant historical information, making the evolutionary patterns less discriminative, as both normal and anomalous nodes would deviate significantly from their distant historical states.

6.4.2. Embedding dimension

In this experiment, we study the impact of embedding dimension d. We vary d within the range of {16, 32, 64, 128, 256}. The results are shown in Fig. 6(b). As shown in this figure, the performance generally improves with the increase of d, underscoring the importance of a larger embedding size for capturing complex and sufficient semantic information. However, when $d \geq 128$, further increasing d does not yield substantial performance gains, and even result in a notable performance decline on the Bitcoin-Alpha datasets. This may be attributed to the fact that too large d make MHisCL hard to converge and prone to overfitting. In most cases, MHisCL achieves superior performance with an embedding dimension of 128.

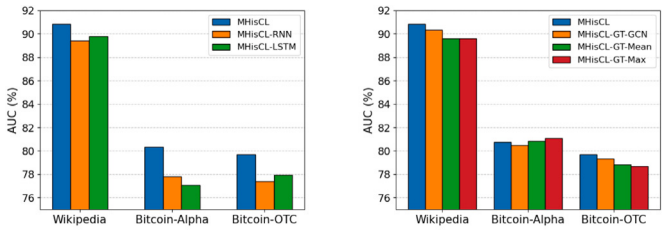
6.4.3. Hyperparameters of contrastive losses

In this experiment, we explore how the hyperparameters of contrastive losses affect the performance of MHisCL, including the temperature τ and the weight α for temporal structural contrastive loss $\mathcal{L}_{\text{tsc}}$. We set τ and α in the range of {0.01, 0.02, 0.05, 0.1, 0.2} and {0.1, 0.5, 1, 2}, respectively. The results are illustrated in Fig. 7. According to Fig. 7, we can see that MHisCL generally benefits from smaller τ and achieves notable performance improvement when $\tau = 0.02$. This trend aligns with the findings in [43], which suggest that a smaller τ allows the model to better distinguish hard negatives—those historical states that are anomalous but exhibit highly similar to the current node state. This enhances the model's ability to discern fine-grained differences between normal and anomalous evolutionary patterns, improving anomaly detection performance. For α, we observe that in the Bitcoin-Alpha dataset, the performance improves significantly as α increases, with a slight decline occurring when $\tau \leq 0.02$ and $\alpha > 1$, indicating the importance of integrating normal structural patterns for effective anomaly detection. In contrast, the Wikipedia dataset exhibits minimal performance fluctuations in response to changes in α, possibly due to the greater importance of temporal patterns for anomaly detection in this dataset. This suggests that the impact of capturing normal structural patterns varies across different datasets.

6.5. Comparison with different dataset splits

To answer **RQ4**, we evaluate the performance of MHisCL under different dataset splits with limited training data. Specifically, we vary the training ratio p from 0.7 to 0.5 with a step size of 0.05, retaining the subsequent 0.15 of the edges as the validation set, and assess the model's performance on the remaining edges. We conduct experiments on the Wikipedia and Bitcoin-Alpha datasets and compare MHisCL with three best-performing baseline models: SLADE, SAD, and TGN. The comparison results are presented in Fig. 8.

From this figure, we can observe that as the training ratio increases, the AUC values generally improve across both datasets, underscoring the importance of sufficient training data for better performance. The observed performance fluctuation, such as at the 0.55 training ratio on the Bitcoin-Alpha dataset, may be attributed to potential shifts in data distribution when expanding the training set. Additionally, as the training ratio p decreases, MHisCL still outperforms all baseline models in most cases and exhibits minimal performance degradation. For example, when p decreases from 0.7 to 0.5, MHisCL experiences only a slight drop in performance, from 90.85% to 89.05%, while the second-best model, SLADE, drops from 88.68% to 86.84%. Notably,



(a) MHisCL variants with different RNNs (b) MHisCL variants with different GNNs

Fig. 9. Performance of MHisCL variants with different RNNs and GNNs.

MHisCL still achieves competitive performance even with a lower training ratio of 0.5. This demonstrates the robustness and effectiveness of MHisCL in detecting anomalies with limited training data. We attribute this to the multi-step histories-based contrastive learning framework, which leverages abundant nodes' multi-step historical information as self-supervision signals, effectively mitigating the impact of data scarcity.

6.6. Comparison with different encoders

To answer **RQ5**, we replace the two major encoders in MHisCL with alternative architectures to examine how different encoders influence overall performance. Specifically, we substitute GRU and GT-GAT in MHisCL with different recurrent neural networks (RNNs) and graph neural networks (GNNs), respectively.

6.6.1. Comparison with different recurrent neural networks

RNN, GRU, and LSTM are three widely used variants of recurrent neural networks for sequence modeling tasks. To explore their impact on the final performance, we replace the GRU in the history recurrent network of MHisCL with RNN and LSTM, respectively, resulting in the variants MHisCL-RNN and MHisCL-LSTM. Notably, LSTM incorporates an additional cell state alongside the hidden state, which we initialize as an all-zero matrix. The comparison results are illustrated in Fig. 9(a). From this figure, we can observe that, the vanilla MHisCL-RNN generally underperforms in most cases due to its intrinsic limitations, such as vanishing gradients. In contrast, MHisCL-GRU and MHisCL-LSTM generally achieve superior performance, highlighting the importance of capturing long-term temporal dependencies. Furthermore, MHisCL using GRU achieves the best performance across all datasets. We attribute this to its simpler yet effective architecture, which allows it to learn temporal dependencies more efficiently while being less prone to overfitting compared to LSTM.

6.6.2. Comparison with different graph neural networks

In this subsection, we integrate SSTE into other two prominent GNNs, namely GCN [44] and GraphSAGE [45], to explore the generalization capability of SSTE across different GNN architectures. Specifically, we substitute their static message during neighborhood aggregation with the temporal messages defined in Eq. (8) to capture rich temporal information. For GraphSAGE, we employ mean and max aggregation functions to aggregate neighborhood temporal messages, respectively. We denote these variants as MHisCL-GT-GCN, MHisCL-GT-Mean, and MHisCL-GT-Max, respectively. The comparison results are illustrated in 9(b), and we summarize the following observations:

First, the attention-based MHisCL generally perform the best, highlighting the significance of selectively attending to neighboring nodes based on their temporal importance. However, the variants using other GNN architectures still achieve competitive performance across all datasets, demonstrating the strong generalization capability of SSTE across diverse GNN architectures. **Second**, MHisCL-GT-GCN, which substitutes the temporal attention weights with static structural weights during neighborhood aggregation, can still achieves highly comparable performance compared to MHisCL across all datasets. This indicates that SSTE can effectively integrates temporal information into the temporal messages, without relying on additional attention mechanisms to explicitly capture temporal similarities. **Third**, MHisCL-GT-Mean and MHisCL-GT-Max generally exhibit inferior performance. This may be because that mean pooling treats all neighboring nodes equally, overlooking their different structural and temporal importance, while max pooling focuses on retaining extreme values, which may lead to information loss. However, it is noted that these two simple variants even outperform attention-based MHisCL on the Bitcoin-Alpha dataset. We attribute this to the relatively simple network structure of Bitcoin-Alpha, where these non-parameterized aggregation functions can capture its structural and temporal information more efficiently. Furthermore, we attribute the best performance of MHisCL-GT-Max to its strong capability to preserve anomalous information that deviates significantly from the overall neighborhood.

7. Conclusion and future work

In this paper, we focus on anomaly detection in continuous-time dynamic graphs. Building upon the empirical observation that normal nodes exhibit smooth long-term evolutionary patterns, while anomalous nodes significantly deviate from these patterns, we propose MHisCL, a multi-step histories-based contrastive learning framework, for self-supervised anomaly detection in dynamic graphs. Extensive experimental results demonstrate a significant performance improvement as the number of time steps for contrastive learning increases, highlighting the effectiveness of capturing smooth long-term evolutionary patterns for self-supervised anomaly detection in dynamic graphs. Furthermore, MHisCL incorporating state space time encoding (SSTE), significantly outperforms its variant with functional time encoding (FTE). This suggests that SSTE, which employs a linear dynamical system to explicitly model the dynamic evolution of node representations over time, can better capture the complex evolving dynamics in dynamic graphs.

Despite the success of MHisCL, there remain areas for further exploration. In future work, we plan to extend MHisCL to other types of graphs, such as discrete-time dynamic graphs and dynamic heterogeneous graphs. Additionally, while we focus on node-level anomalies, there are various types of anomalies at different scales in graphs, such as subgraph-level and community-level anomalies. We aim to explore anomaly detection at these larger scales in future work.

CRediT authorship contribution statement

Yun Fu: Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Che Zhou:** Writing – review & editing, Validation, Data curation. **Jintang Li:** Writing – review & editing, Validation, Investigation. **Liang Chen:** Writing – review & editing, Supervision, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The research is supported by the National Key R&D Program of China under grant No. 2022YFF0902500, the Guangdong Basic and Applied Basic Research Foundation, China (No. 2023A1515011050), and Shenzhen Science and Technology Program (KJZD20231023093450 1003).

Data availability

Data will be made available on request.

References

[1] S. Tian, J. Dong, J. Li, W. Zhao, X. Xu, B. Wang, B. Song, C. Meng, T. Zhang, L. Chen, SAD: Semi-supervised anomaly detection on dynamic graphs, in: IJCAI, ijcai.org, 2023, pp. 2306–2314.

[2] Y. Liu, S. Pan, Y.G. Wang, F. Xiong, L. Wang, Q. Chen, V.C. Lee, Anomaly detection in dynamic graphs via transformer, IEEE Trans. Knowl. Data Eng. 35 (12) (2023) 12081–12094.

[3] D. Eswaran, C. Faloutsos, S. Guha, N. Mishra, SpotLight: Detecting anomalies in streaming graphs, in: KDD, ACM, 2018, pp. 1378–1386.

[4] K. Ding, J. Li, R. Bhanushali, H. Liu, Deep anomaly detection on attributed networks, in: SDM, SIAM, 2019, pp. 594–602.

[5] J. Li, H. Dani, X. Hu, H. Liu, Radar: Residual analysis for anomaly detection in attributed networks, in: IJCAI, ijcai.org, 2017, pp. 2152–2158.

[6] D. Wang, Y. Qi, J. Lin, P. Cui, Q. Jia, Z. Wang, Y. Fang, Q. Yu, J. Zhou, S. Yang, A semi-supervised graph attentive network for financial fraud detection, in: ICDM, IEEE, 2019, pp. 598–607.

[7] K. Ding, Q. Zhou, H. Tong, H. Liu, Few-shot network anomaly detection via cross-network meta-learning, in: WWW, ACM/IW3C2, 2021, pp. 2448–2456.

[8] S.M. Kazemi, R. Goel, Representation learning for dynamic graphs: A survey, J. Mach. Learn. Res. 21 (70) (2020) 1–73.

[9] W. Yu, W. Cheng, C.C. Aggarwal, K. Zhang, H. Chen, W. Wang, NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks, in: KDD, ACM, 2018, pp. 2672–2681.

[10] L. Zheng, Z. Li, J. Li, Z. Li, J. Gao, AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN, in: IJCAI, ijcai.org, 2019, pp. 4419–4425.

[11] L. Cai, Z. Chen, C. Luo, J. Gui, J. Ni, D. Li, H. Chen, Structural temporal graph neural networks for anomaly detection in dynamic graphs, in: CIKM, ACM, 2021, pp. 3747–3756.

[12] J. Guo, S. Tang, J. Li, K. Pan, L. Wu, RustGraph: Robust anomaly detection in dynamic graphs by jointly learning structural-temporal dependency, IEEE Trans. Knowl. Data Eng. (2023).

[13] D. Chen, X. Zhao, W. Xiao, Fine-grained anomaly detection on dynamic graphs via attention alignment, in: 2024 IEEE 40th International Conference on Data Engineering, ICDE, IEEE, 2024, pp. 3178–3190.

[14] D. Xu, C. Ruan, E. Körpeoglu, S. Kumar, K. Achan, Inductive representation learning on temporal graphs, in: ICLR, OpenReview.net, 2020.

[15] J. Lee, S. Kim, K. Shin, SLADE: Detecting dynamic anomalies in edge streams without labels via self-supervised learning, 2024, CoRR abs/2402.11933.

[16] C.C. Aggarwal, Y. Zhao, S.Y. Philip, Outlier detection in graph streams, in: 2011 IEEE 27th International Conference on Data Engineering, IEEE, 2011, pp. 399–409.

[17] S. Ranshous, S. Harenberg, K. Sharma, N.F. Samatova, A scalable approach for outlier detection in edge streams using sketch-based approximations, in: Proceedings of the 2016 SIAM International Conference on Data Mining, SIAM, 2016, pp. 189–197.

[18] A.K. Ghoshal, N. Das, Anomaly detection in evolutionary social networks leveraging community structure, in: 2021 IEEE International Conference on Service Operations and Logistics, and Informatics, SOLI, IEEE, 2021, pp. 1–6.

[19] A.K. Ghoshal, N. Das, S. Das, A fast community-based approach for discovering anomalies in evolutionary networks, in: 2022 14th International Conference on COMmunication Systems & NETworkS, COMSNETS, IEEE, 2022, pp. 455–463.

[20] A. Dey, B.R. Kumar, B. Das, A.K. Ghoshal, Outlier detection in social networks leveraging community structure, Inform. Sci. 634 (2023) 578–586.

[21] J. Li, R. Wu, W. Sun, L. Chen, S. Tian, L. Zhu, C. Meng, Z. Zheng, W. Wang, What's behind the mask: Understanding masked graph modeling for graph autoencoders, in: KDD, ACM, 2023, pp. 1268–1279.

[22] P. Velickovic, W. Fedus, W.L. Hamilton, P. Liò, Y. Bengio, R.D. Hjelm, Deep graph infomax, in: ICLR (Poster), vol. 2, 2019, p. 4, 3.

[23] D. Xu, W. Cheng, D. Luo, H. Chen, X. Zhang, InfoGCL: Information-aware graph contrastive learning, in: NeurIPS, 2021, pp. 30414–30425.

[24] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, in: ICML Workshop on Graph Representation Learning and beyond, 2020.

[25] K. Hassani, A.H.K. Ahmadi, Contrastive multi-view representation learning on graphs, in: ICML, vol. 119, PMLR, 2020, pp. 4116–4126.

[26] S. Thakoor, C. Tallec, M.G. Azar, R. Munos, P. Veličković, M. Valko, Bootstrapped representation learning on graphs, in: ICLR 2021 Workshop on Geometrical and Topological Representation Learning, 2021.

[27] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, G. Karypis, Anomaly detection on attributed networks via contrastive self-supervised learning, IEEE Trans. Neural Netw. Learn. Syst. 33 (6) (2022) 2378–2392.

[28] J. Duan, S. Wang, P. Zhang, E. Zhu, J. Hu, H. Jin, Y. Liu, Z. Dong, Graph anomaly detection via multi-scale contrastive learning networks with augmented view, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, 2023, pp. 7459–7467, 6.

[29] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, K. Achan, Self-attention with functional time representation learning, in: NeurIPS, 2019, pp. 15889–15899.

[30] A. Rahimi, B. Recht, Random features for large-scale kernel machines, Adv. Neural Inf. Process. Syst. 20 (2007).

[31] Y.-H.H. Tsai, S. Bai, M. Yamada, L.-P. Morency, R. Salakhutdinov, Transformer dissection: A unified understanding of transformer's attention via the lens of kernel, 2019, arXiv preprint arXiv:1908.11775.

[32] A. Gu, T. Dao, S. Ermon, A. Rudra, C. Ré, HiPPO: Recurrent memory with optimal polynomial projections, in: NeurIPS, 2020.

[33] A. Gu, K. Goel, C. Ré, Efficiently modeling long sequences with structured state spaces, in: ICLR, OpenReview.net, 2022.

[34] J.T.H. Smith, A. Warrington, S.W. Linderman, Simplified state space layers for sequence modeling, in: ICLR, OpenReview.net, 2023.

[35] E. Nguyen, K. Goel, A. Gu, G.W. Downs, P. Shah, T. Dao, S. Baccus, C. Ré, S4ND: Modeling images and videos as multidimensional signals with state spaces, in: NeurIPS, 2022.

[36] G. Klambauer, T. Unterthiner, A. Mayr, S. Hochreiter, Self-normalizing neural networks, in: NIPS, 2017, pp. 971–980.

[37] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: ICLR (Poster), OpenReview.net, 2018.

[38] S. Kumar, X. Zhang, J. Leskovec, Predicting dynamic embedding trajectory in temporal interaction networks, in: KDD, ACM, 2019, pp. 1269–1278.

[39] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, M. Bronstein, Temporal graph networks for deep learning on dynamic graphs, in: ICML 2020 Workshop on Graph Representation Learning, 2020.

[40] S. Kumar, F. Spezzano, V.S. Subrahmanian, C. Faloutsos, Edge weight prediction in weighted signed networks, in: ICDM, IEEE Computer Society, 2016, pp. 221–230.

[41] J.J. McAuley, J. Leskovec, From amateurs to connoisseurs: modeling the evolution of user expertise through online reviews, in: Proceedings of the 22nd International Conference on World Wide Web, 2013, pp. 897–908.

[42] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos, V. Subrahmanian, Rev2: Fraudulent user prediction in rating platforms, in: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, 2018, pp. 333–341.

[43] F. Wang, H. Liu, Understanding the behaviour of contrastive loss, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 2495–2504.

[44] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.

[45] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: NeurIPS, 2017, pp. 1024–1034.